How similar are two images?

Programming and Tools for AI – Week 8

James McDermott





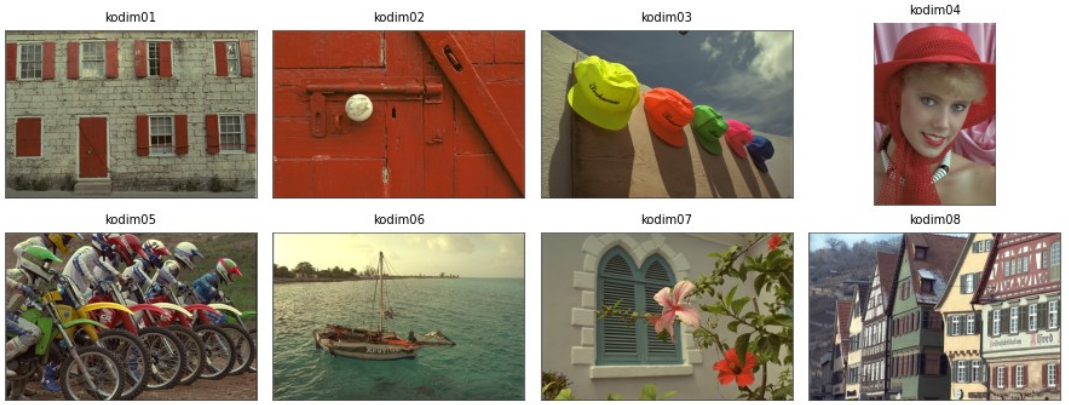


Part 1: the problem and the data

How similar are two images?

Problem. A folder holds photographs. Some are similar, and some are actually the same picture saved at different sizes. Given any one photo — the **query photo** — our goal is to find the ones most similar to it.

We start with the **Kodak True Color** suite with 24 photos. We will use only the image data (no labels, captions, or filenames).



The same picture, three sizes

full
512 x 768 x 3



half
256 x 384 x 3



thumb
64 x 96 x 3



A photo at three different sizes: 1.2M floats, v 300k, v 18k. We cannot compare the arrays directly to determine similarity.

Where this question comes up

- **Reverse image search.** You upload a picture and get the pages it appears on.
- **De-duplicating a dataset.** The same image occurring in train and test will “cheat” an image classification task, so we should remove it.
- **Copyright and provenance.** Has this photo been published before, cropped or resized?
- **Your phone’s photo library.** “Here are 14 near-identical shots of the same cat; keep one?”

What would we like “similar” to mean?

Let’s think about what we want.

- The query photo and its own thumbnail should be very close according to our method.
- Two photos of a red door should be closer than a red door and a seascape.
- It should not depend on image size, since we have three of each.
- It should be a **number**, so we can sort by it.
- We said “similar”, but in fact we will create a number that measures **dissimilarity** or **distance**. When it’s zero, the photos should be effectively identical.

Our plan

1. **Read** each photo into an array of numbers.
2. **Summarise** each photo as a short list of numbers — a fixed length, whatever the photo's size.
3. **Compare** the query photo's summary against all the others, using plain old Euclidean distance, and sort.

Reading a photo to a Numpy array

A photo is a large 3d array (height, width, colour channel).

When dealing with large amounts of numeric data, **numpy** is the right tool.

```
import numpy as np
import matplotlib.pyplot as plt

image = plt.imread("kodim01_full.png")
>>> image.shape
(512, 768, 3)
>>> type(image)
<class 'numpy.ndarray'>
>>> image.ndim # equal to len(image.shape)
3
```

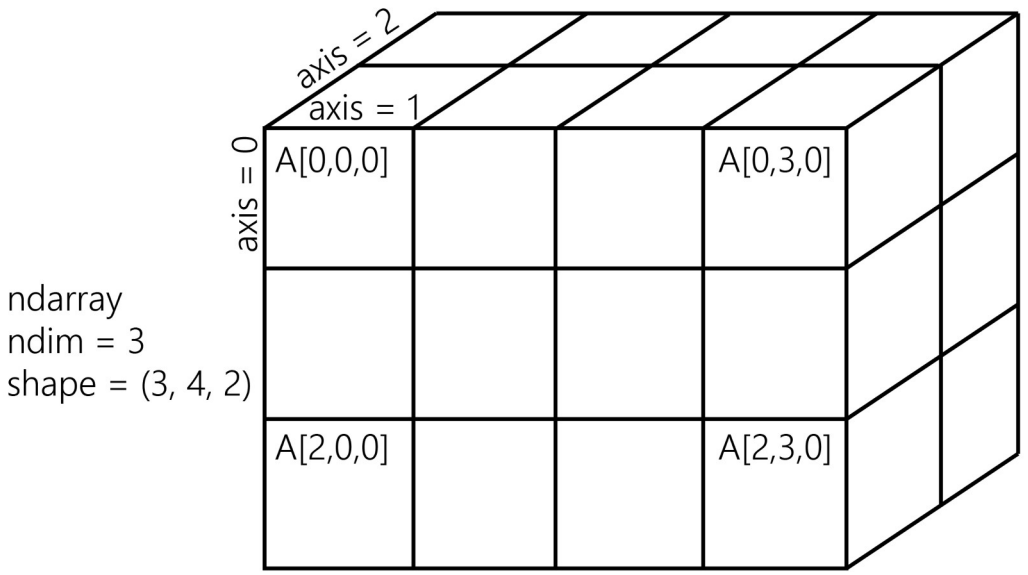
512 rows, 768 columns, and 3 numbers per position: red, green, blue

shape and axes

An `ndarray`, also known as just `array`, has a `shape`: a tuple with one entry per **axis**.

- axis 0: height, 512 rows
- axis 1: width, 768 columns
- axis 2: colour, 3 channels

`ndim` or `len(shape)` is the number of axes.

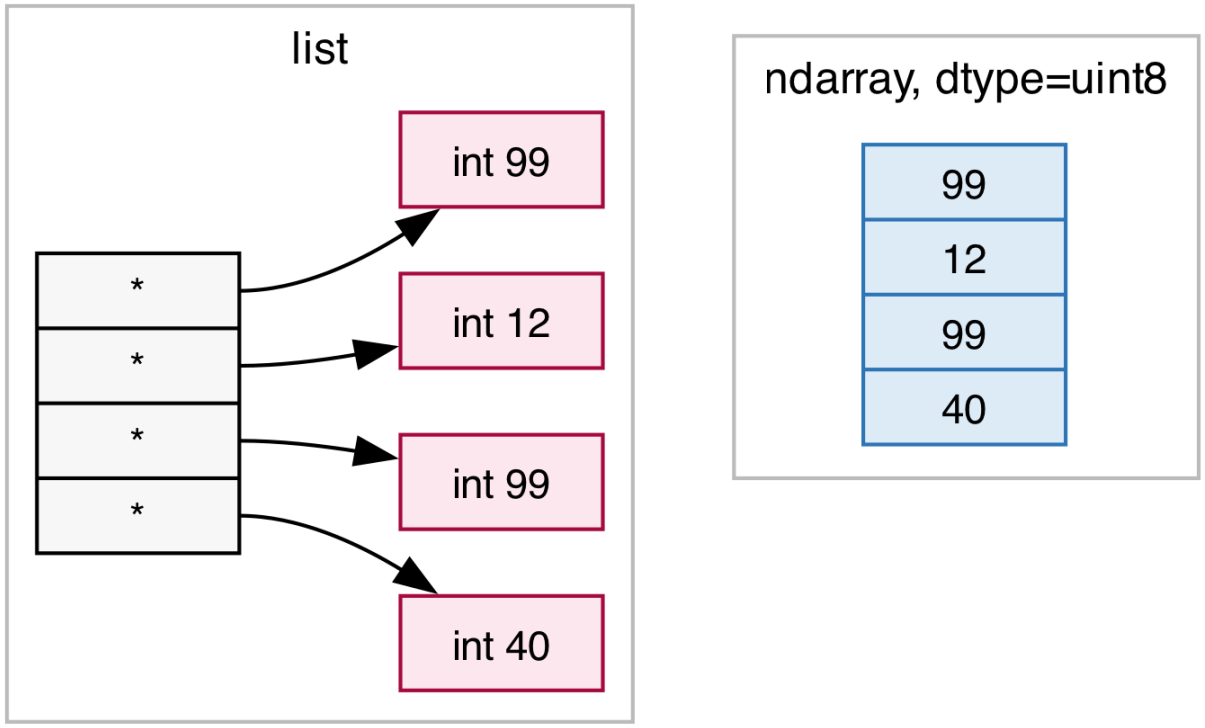


dtype: every element has the same type

```
>>> image.dtype
dtype('uint8')
>>> image.size
1179648
>>> image.nbytes
1179648
```

`uint8` is an **unsigned 8-bit integer**, 0 to 255. One byte each, and `nbytes == size` exactly. A Python list of a million ints would be around 36 MB for the same data because it stores a pointer per int, and every int is an object.

One type, one block of memory



A list holds pointers to objects which ‘wrap’ ints. An array holds just the ints themselves.

Indexing and slicing

We can use a **tuple of indices**, one value per axis. Slices work exactly as they do on lists.

```
>>> image[0, 0]                    # top-left pixel, all 3 channels
array([99, 99, 99], dtype=uint8)
>>> image[0, 0, 1]                 # its green value
99
>>> image[:, :, 0].shape           # the red channel of the whole
image
(512, 768)
>>> image[:100, :100].shape        # a 100x100 crop
(100, 100, 3)
```

Writing to an array

A slice of an array is a **view**, not a copy: write to it and you change the original.

```
>>> image[:100, :100] = 0      # set the top-left corner to black
```

Use `.copy()` when you want a slice you can change safely.

Reading a photo

```
def load(path):
    # drop alpha if present
    rgb = plt.imread(path)[:, :, :3]
    # 0.0–1.0 becomes 0–255
    return np.round(rgb * 255).astype(np.uint8)
```

We pre-process in two ways:

- `plt.imread` gives floats from 0.0 to 1.0. We will scale to give 0-255 integers.
- A PNG may have a fourth **alpha** (transparency) channel. We drop it using `[:, :, :3]`.

Finding the files (not examinable)

```python
from pathlib import Path

DATA = Path(__file__).parent / "data"
paths = sorted((DATA / "photos").glob("*.png"))
```

`glob` returns an iterator of `Path` objects; `sorted` pins the order down so that two runs agree. `Path(__file__).parent` means the file works whatever directory you run it from.

The photos are not in the weekly zip. `prep_data.py` fetches them.

```
python prep_data.py
```

Part 2: counting the pixels

Summarising a photo by its colours

Step 2 is to summarise a photo by a list of numbers, ie a vector.

Here is a simple idea. Forget *where* the colours are and count *how much* of each colour there is. A photo of a red door will be mostly red whatever its size; a seascape will be mostly blue-green.

“How much of each colour” is called a **colour histogram**.

Too many colours

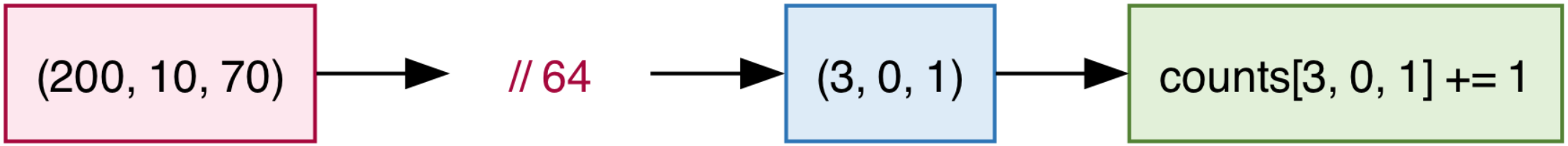
A pixel has 256 possible reds, 256 greens, 256 blues: 16.7 million distinct colours. A 512x768 photo has only 393 thousand pixels, so almost every colour never occurs.

So we **bin**. For now we use B=4 bins per channel, each one 256 // B = 64 wide:

```
0–63   ->  0          128–191  ->  2
64–127 -> 1           192–255  ->  3
```

This is just integer division: `value // 64`.

4 x 4 x 4 = 64 bins



Given B=4, a pixel with red 200, green 10, blue 70 lands in bin (3, 0, 1). Later we can try different B.

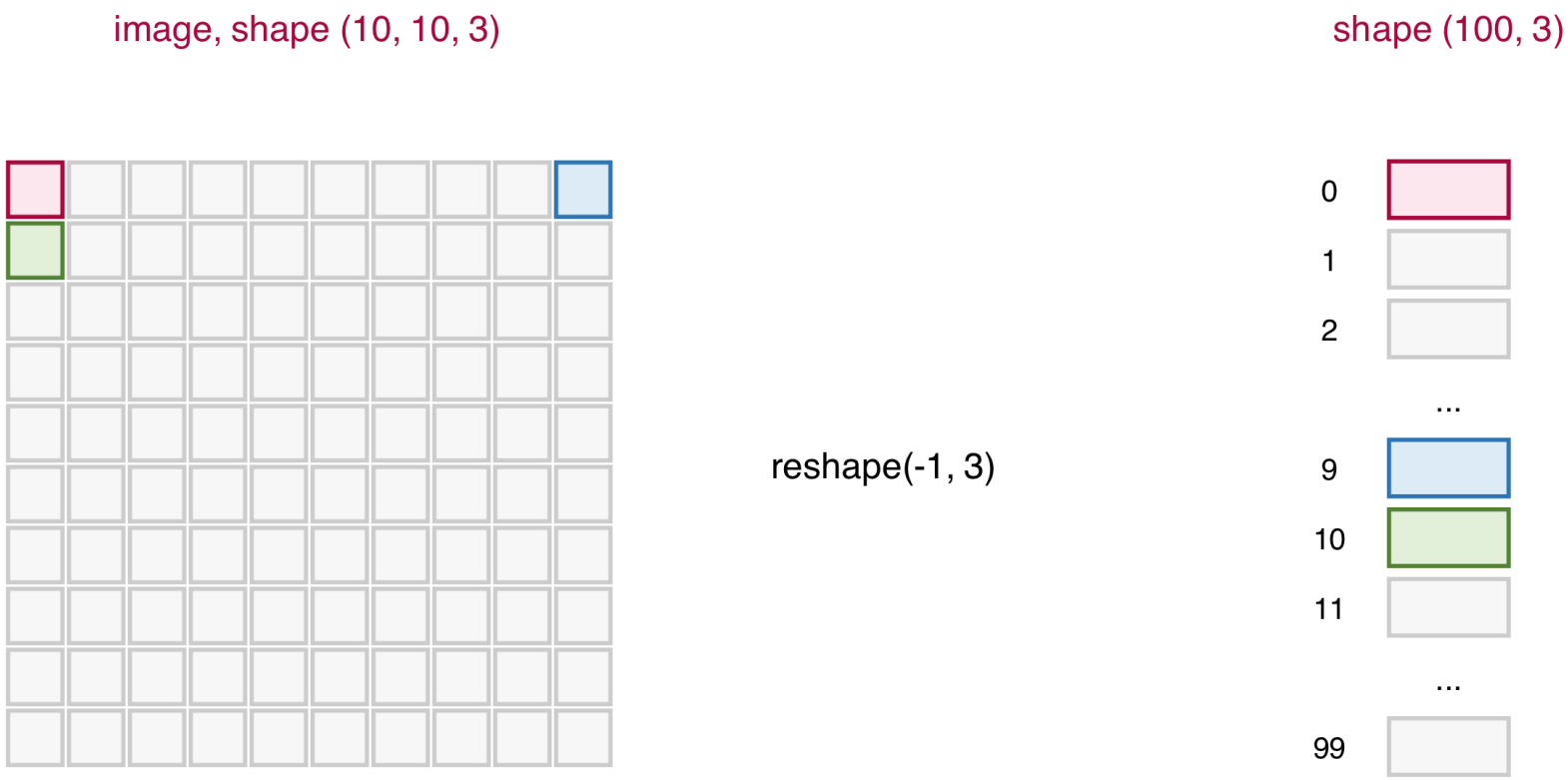
The counts, in plain Python

```
def histogram_python(image):
    counts = Counter()
    for red, green, blue in image.reshape(-1, 3):
        counts[red // WIDTH, green // WIDTH, blue // WIDTH] += 1
    return np.array([[[counts[r, g, b] for b in range(B)]
                      for g in range(B)] for r in range(B)])
```

`image.reshape(-1, 3)` forgets the height and width and leaves a plain list of pixels, each pixel shaped `(3,)`. The `-1` means “work out this one from the total”.

We say “plain Python” because we are using the Numpy `np.array` just to wrap up the final result, but we calculated the final result using plain Python.

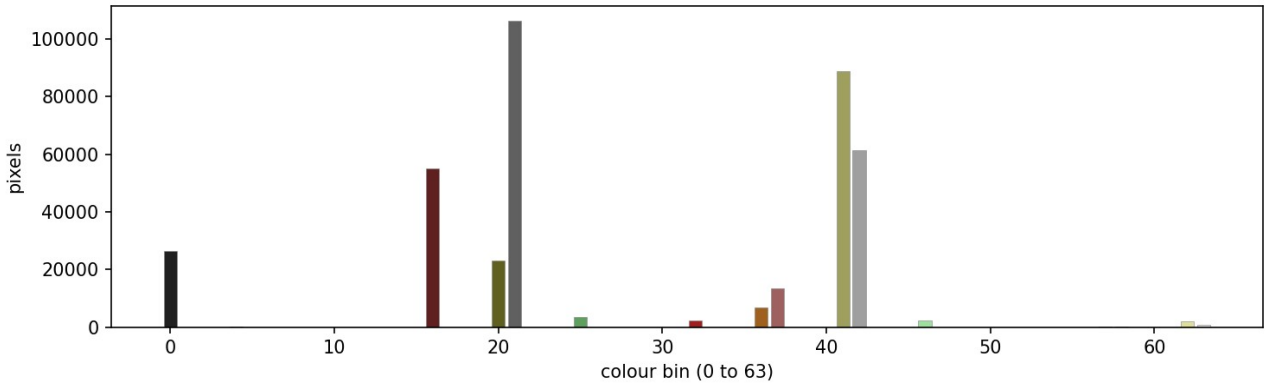
reshape



reshape does not truly reorder, as that would be slow; it relabels a block of memory. It reads the pixels left to right and then top to bottom, so pixel (0,0) is element 0, pixel (0,9) is element 9, and pixel (1,0) is element 10. The -1 means: work this one out from the total.

What our algorithm produces

```
>>> counts =
histogram_python(image)
>>> counts.shape
(4, 4, 4)
>>> counts.sum()              #
every pixel landed somewhere
393216
```



Each bar is drawn in the colour of its own bin. This is kodim01: a lot of grey stone, a little red.

Comparing to a number gives an array

```
>>> x = np.array([1, 5, 3, 9, 2])
>>> x > 2
array([False,  True,  True,  True, False])
>>> np.sum(x > 2)
3
>>> x[x > 2]
array([5, 3, 9])
```

A **boolean mask**: same shape as the array, one `True` or `False` per element. Summing it counts the `True`s. Indexing with it selects them.

The counts again, one bin at a time

```
def histogram_numpy(image):
    binned = image // WIDTH   # every value is now 0, 1, 2 or 3
    counts = np.zeros((B, B, B), dtype=int)
    for r in range(B):
        for g in range(B):
            for b in range(B):
                is_r = binned[:, :, 0] == r
                is_g = binned[:, :, 1] == g
                is_b = binned[:, :, 2] == b
                counts[r, g, b] = np.sum(is_r & is_g & is_b)
    return counts
```

The loops run over the 4x4x4 bins, not over the 393k pixels. Each `is_r` is a whole-image map of `True` and `False`.

There is still a loop

Both functions do the same arithmetic. What changed is **who runs the loop**: Python, one pixel at a time, or Numpy, in C, over a whole array.

```
>>> np.array_equal(histogram_numpy(image), histogram_python(image))
True
```

This is an example of a good pattern: start in Python with logic you understand, then rewrite something in Numpy, and check the new version gives the same result.

Part 3: comparing the photos

One row per photo

	bin 0	bin 1	...	bin 63
kodim01_full	26433	0	...	852
kodim01_half	5440	0	...	182
...	...	...	...	...
kodim24_thumb	574	0	...	349

Each 4x4x4 becomes a row of 64 numbers. Now put all rows together. X.shape is (72, 64). From here we just treat it as table of features. We no longer use the photo data.

Comparing two rows: Euclidean distance

Each row is just a point in 64 dimensions, so we use ordinary Euclidean distance between two vectors: square the differences, add them up, take the square root.

We met `math.dist` in Week 7. This is the same thing but now in Numpy.

```
def euclidean_dist(a, b):
    return np.sqrt(np.sum((a - b)**2))
```

`np.linalg.norm` is a shortcut which takes in a single vector. We just need to subtract manually to get the differences, and pass that in.

```
np.linalg.norm(a - b)  # same as euclidean_distance(a, b)
```

axis=

A method like `.sum()` is a **summarising method**. It takes an `axis` argument. Think of it as the axis that **disappears** in the summary.

```
>>> X = np.array([[0, 1, 2],
...               [3, 4, 5]])
>>> X.sum(axis=0)     # down the columns; the rows vanish
array([3, 5, 7])
>>> X.sum(axis=1)     # along the rows; the columns vanish
array([ 3, 12])
>>> X.sum()           # everything
15
```

Many distances at once

We want all distances from our query photo to every photo in the dataset. In Python that would mean writing a loop and calculating a distance at each iteration. Numpy makes the code shorter and faster:

```
def distances_to(X, x):
    return np.linalg.norm(X - x, axis=1)

>>> X = np.array([[0.0, 0.0], [3.0, 4.0], [6.0, 8.0]])
>>> distances_to(X, X[0])
array([ 0.,  5., 10.])
```

NB X and x are **different shapes** so this requires **broadcasting** which we will see shortly.

argsort: the order, not the values

```
>>> d = np.array([0.4, 0.1, 0.9])
>>> np.argsort(d)
array([1, 0, 2])
```

“The smallest is at position 1, then position 0, then position 2.”

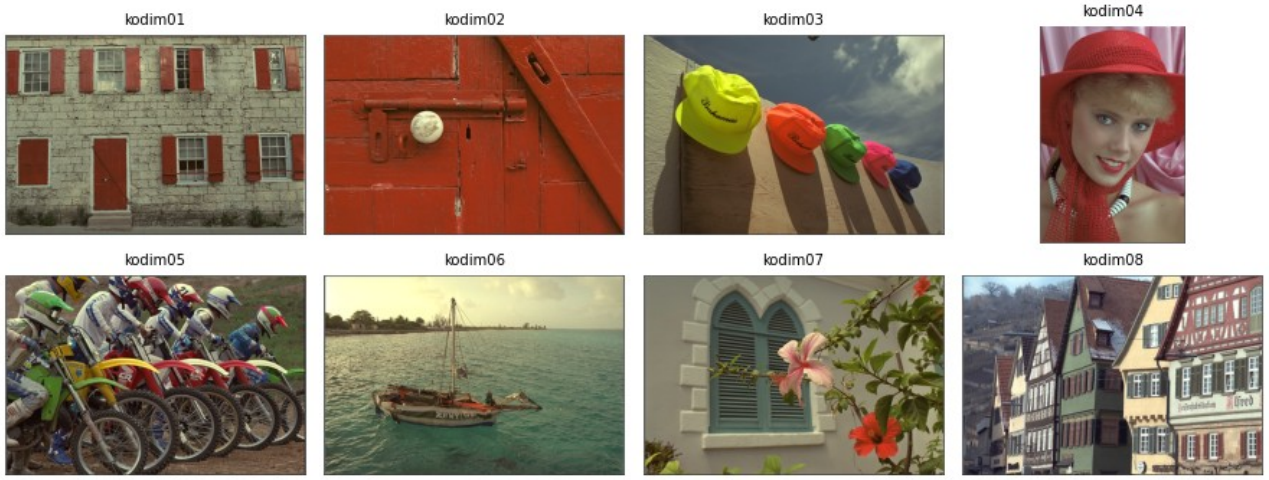
```
def most_similar(X, i, k=3):
    d = distances_to(X, X[i])
    d[i] = np.inf  # the query photo is not its own neighbour
    return np.argsort(d)[:k]
```

Suppose we have an array `names`. Then we can use `names[most_similar(X, i)]`, indexing one array with an array of positions. That is **fancy indexing**.

Try it

```
nearest neighbours of
kodim01_thumb

  raw counts:
['kodim07_thumb'
'kodim03_thumb'
'kodim06_thumb']
```



A stone house matched to a flower, some baseball caps and a boat. What went wrong?

What went wrong

Look at what the three answers have in common: they are all thumbnails.

A thumbnail has 18k pixels and a full-size photo has 393k, so every count in a thumbnail row is much smaller. The distance is dominated by **how many pixels there are** instead of their colours.

Proportions, not counts

We divide each row by its own total. Now every row sums to 1.

`X.sum(axis=1)` gives 72 totals, shape `(72,)`. Reshaping to `(72, 1)` makes it a column, so each row is divided by its own number.

```
def as_proportions(X):
    return X / X.sum(axis=1).reshape(-1, 1)
```

```
>>> X = np.array([[1.0, 3.0], [10.0, 30.0]])
```

```
>>> as_proportions(X)
```

```
array([[0.25, 0.75],
       [0.25, 0.75]])
```

Two rows that were 10 times apart are now identical.

Try it again

```
  proportions: ['kodim01_half'
'kodim01_full' 'kodim07_half']
```



Its own half-size and full-size versions, then the flower it liked before.

The whole thing

```
paths = sorted((DATA / "photos").glob("*.png"))
names = np.array([p.stem for p in paths])

X = build_matrix(paths)                    # (72, 64) raw counts
P = as_proportions(X)                      # (72, 64) proportions

query = 0
print(names[most_similar(P, query)])
```

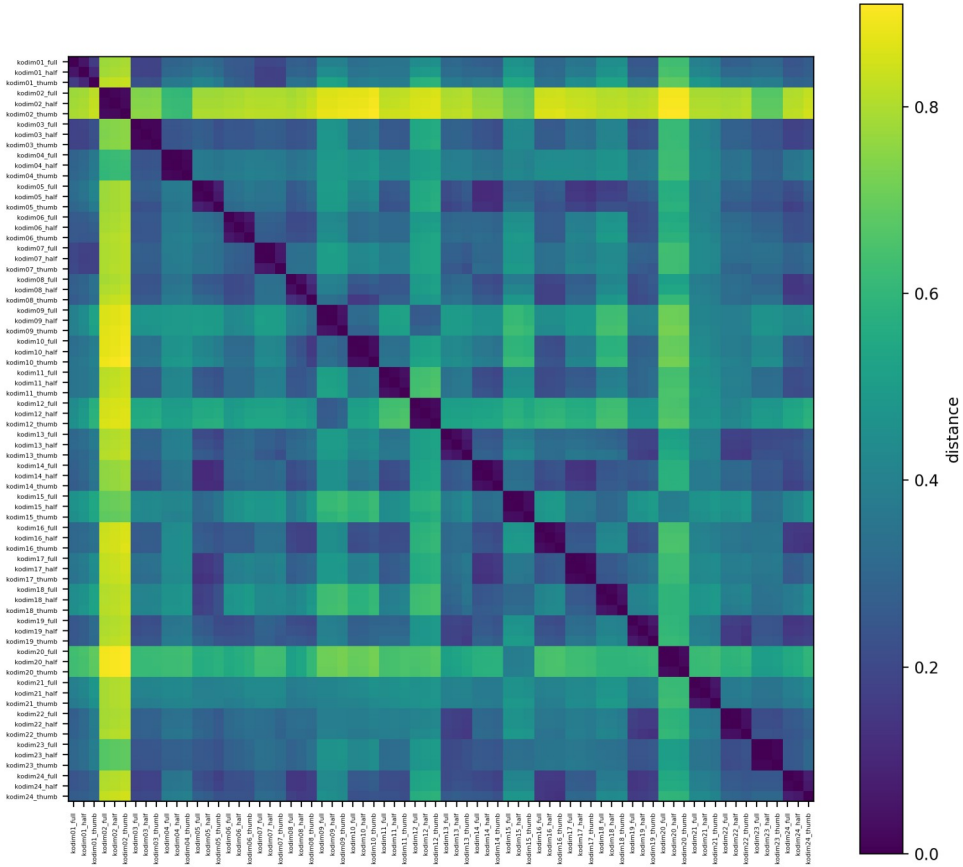
All distance pairs at once

One photo against all the others was one line. All photos against all photos is `cdist`: every row of the first against every row of the second.

```
from scipy.spatial.distance
import cdist

D = cdist(P, P)    # (72, 72)
```

`plt.imshow(D)` draws it as a picture, one pixel per entry. The dark diagonal is every photo's distance to itself — look closely and it is a chain of 3x3 blocks, the full, half and thumbnail versions of one photo.



Part 4: vectorisation and broadcasting

Vectorisation

Arithmetic on an array applies element by element.

```
>>> np.array([1, 2, 3]) * 10
array([10, 20, 30])
```

So a function written for a scalar often works on an array unchanged:

```
def f(x):
    return 2 * x ** 2 - 10
>>> f(3)
8
>>> f(np.array([0, 1, 2, 3]))
array([-10,  -8,  -2,   8])
```

Vectorisation with multiple arrays

Arithmetic on an array applies element by element. If two arrays are the same shape it's easy to understand:

```
>>> x = np.array([1, 2, 3])
>>> y = np.array([10, 20, 30])
>>> x * y
array([10, 40, 90])
```

Broadcasting: vectorisation with different shapes

Broadcasting is how Numpy handles two arrays whose shapes do not match: it stretches length-1 axes and missing axes until they match.

No data is actually copied (which would waste time and space). It “pretends to stretch” with clever internal book-keeping.

We used broadcasting several times already:

```
np.array([1, 2, 3]) * 10                  # array * scalar
image // WIDTH                            # array // scalar
X / X.sum(axis=1).reshape(-1, 1)          # (72, 64) / (72, 1)
X - x                                     # (72, 64) - (64,)
```

The rule

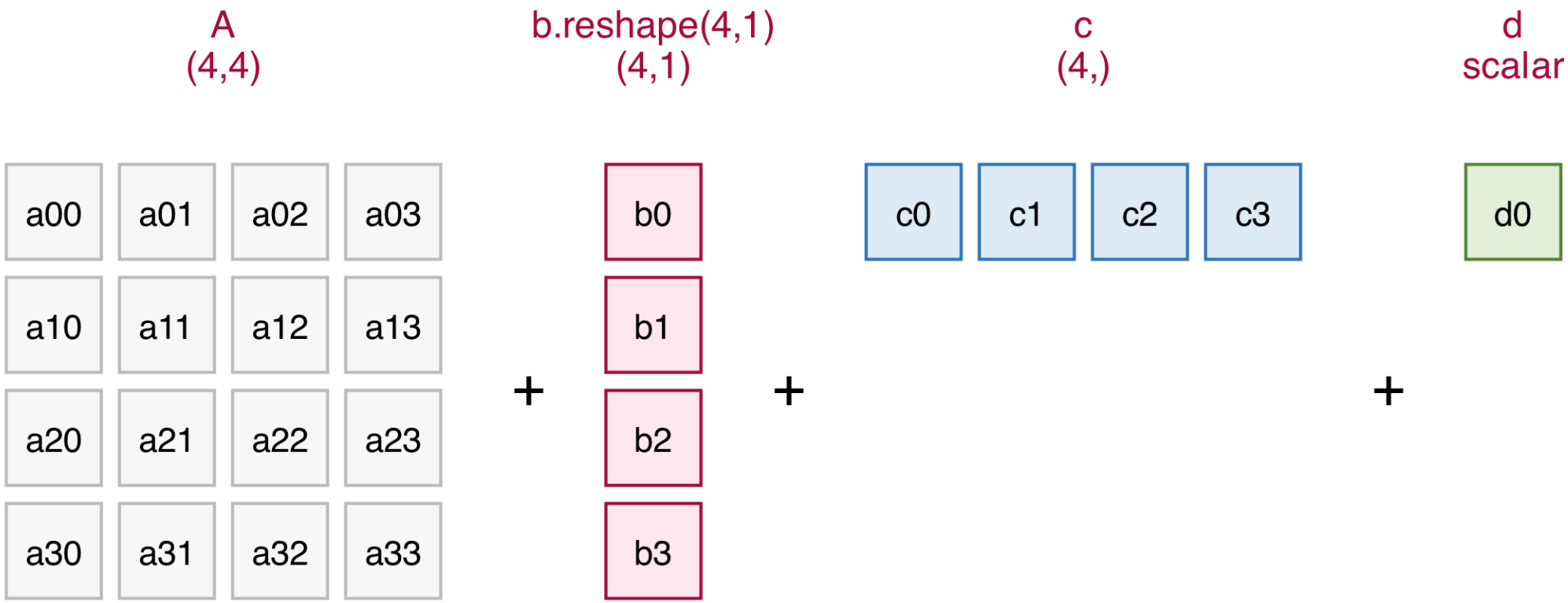
Line the two shapes up at the **right-hand end**, and pad the shorter one with 1s on the left. Two axes are compatible if they are **equal**, or if **either is 1**.

```
(72, 64)
    (64,)  -> ok: result shape (72, 64)


(3, 5)
   (3,)    -> error: `ValueError`

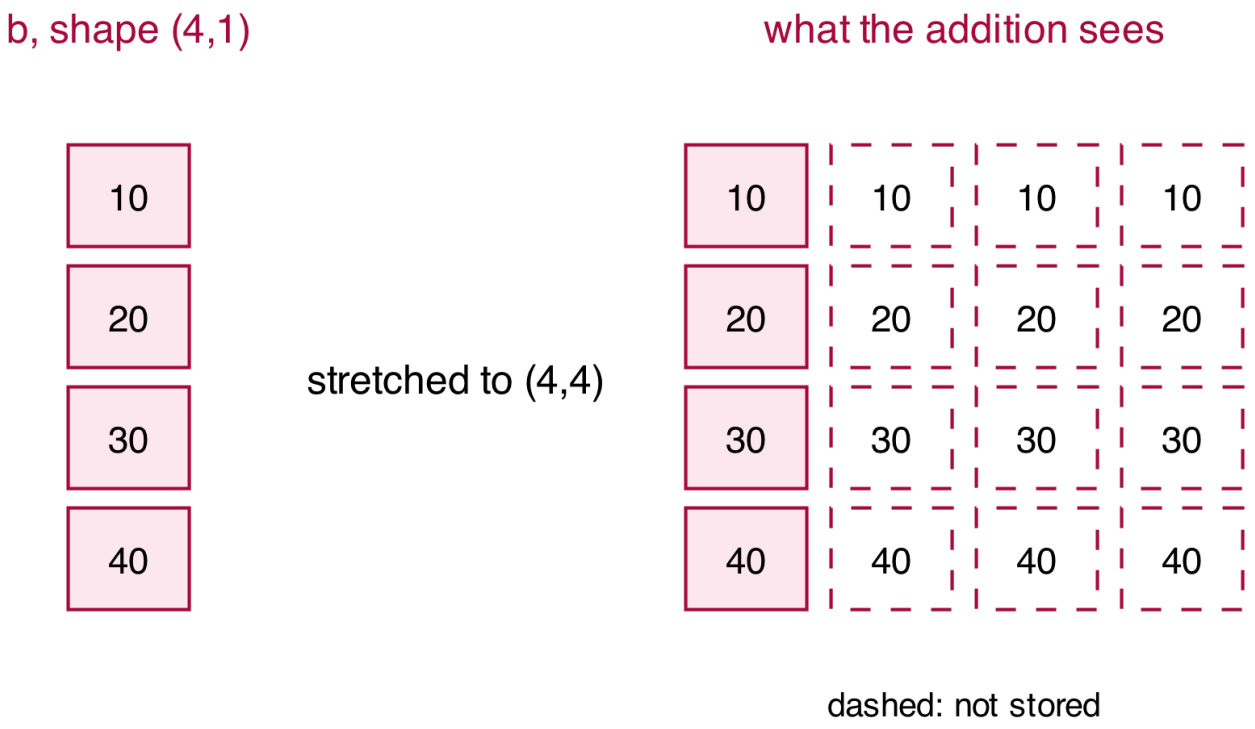

(2, 1, 4)
   (3, 1)  -> ok: result shape (2, 3, 4)
```

Summing four different shapes



Suppose we have A (4, 4), B (4,), C (4,), D (1,). We want `z[i, j] = A[i, j] + b[i] + c[j] + d`. Then we write: `z = A + b.reshape(-1, 1) + c + d`.

Visualising broadcasting



b is four numbers, and it behaves as though it were sixteen. Numpy never writes those sixteen down.

What broadcasting is for

It lets you say what you mean without a loop:

```
X - x                          # subtract one row from all 72
X / totals                     # each row by its own total
```

The @ operator

@ is matrix multiplication, and for two 1-D arrays it is the dot product.

```
>>> np.array([[1., 2.], [3., 4.]]) @ np.array([10., 100.])
array([210., 430.])
```

Eg, a grey pixel is a weighted average of red, green and blue — the eye is far more sensitive to green than to blue:

```
WEIGHTS = np.array([0.2126, 0.7152, 0.0722])
gray = image @ WEIGHTS              # (h, w, 3) @ (3,) -> (h, w)
```

Speedups (not examinable)

```
from time import perf_counter

start = perf_counter()
histogram_python(image)
print(f"{(perf_counter() - start) * 1000:.1f} ms")
```

On one photo, 512 x 768, we get 25x speedup:

```
histogram_python        1736.9 ms
histogram_numpy           70.4 ms
```

Numpy does not change complexity

Both versions look at every pixel once. Both are `O(h * w)`. The complexity **class** did not change, and Numpy does not change it: you cannot count the pixels without visiting the pixels.

But Numpy does give a huge speedup thanks to C implementation which allows faster loops.

- In a previous week we saw a speedup by rewriting an exponential algorithm into a quadratic one.
- Now we get a speedup by using a faster implementation.

But Numpy is much faster

Python's per-element cost, all of which Numpy skips:

- unbox the `int` object and find its value
- look up `__add__` on its type
- allocate a new object for the result
- re-check the loop condition, in the interpreter

Numpy does the arithmetic in a compiled C loop over bytes that are already adjacent in memory. The data type is checked just once at the start.

When Numpy will not help

- **Small arrays.** There is a fixed cost to every Numpy call. Under a few hundred elements, a for-loop can win.
- **Anything non-numeric.** Strings, dicts, objects. Numpy is intended for numerical data.
- **Genuinely sequential work.** Each step depending on the last, as in Newton's method or a graph search.
- **Python loops.** If you write `for i in range(len(x)): y[i] = x[i] * 2` and `x` and `y` are Numpy arrays, this is slow. Numpy wins when we "hide" the loop behind Numpy calls.

Summary

- A photo is a **`(height, width, 3)` array**, so image processing is just arithmetic.
- A **colour histogram** turns a photo of any size into a fixed-length row, and then the photos are a table of numbers. We use **proportions**.
- Numpy is a crucial library for large numerical data.
- A Numpy array has a type, eg `float`, and a shape, eg `(100, 100, 3)`.
- Multi-axis indexing and slicing, boolean masks, fancy indexing;
- **Vectorisation** and **broadcasting** “hide” the loop, so it runs in C behind the scenes.
- Numpy gives speedup but doesn’t change complexity class.

New this week: `numpy` as `np`; `ndarray`, `shape`, `dtype`, `ndim`, `np.array`, `np.zeros`, `reshape`, `axis=`; `np.sum`, `np.argsort`, `np.linalg.norm`, `np.inf`; `@`; `matplotlib.pyplot` `imread`, `imshow`, `plot`, `savefig`; `scipy.spatial.distance.cdist`.